

Instituto Tecnológico de Costa Rica
Escuela de Ingeniería en Computación
Diseño de Software

Tercer Proyecto: Servidor nivel producción en Azure para un juego por turnos

Documentación del servidor ASHENFALL: servidor autoritativo en Azure para un duelo 1v1 por turnos

Estudiante: Kevin Miranda Méndez
Carné 2024120764

Profesor: Jonathan Solis Parajeles

Juego en línea:
https://ashenfall-server-prod.icytree-9cfd11e3.canadacentral.azurecontainerapps.io
Repositorio: https://github.com/kevinmirandaa/ashenfall-game
Versión evaluada: etiqueta v1.0.0

II Semestre 2026

1. Introducción

Este documento describe el **servidor** que construí para el tercer proyecto del curso Diseño de Software: un servidor de nivel producción, desplegado en Azure, para un juego multijugador por turnos. Como pide el enunciado, lo que se evalúa es el servidor (qué tan bien está construido, qué tan seguro y eficiente es y cuánto sigue los estándares de la industria) y no el juego, que es solo el pretexto. Por eso la documentación explica qué servidor se hizo, dónde corre, con qué tecnologías, cómo está configurado, cómo se protege, cómo se despliega y cómo se midió, siguiendo el orden de los requisitos del enunciado. El juego se resume en el Anexo A.

El servidor se llama **ASHENFALL** y atiende duelos 1v1 por turnos. Es autoritativo: el cliente solo envía intenciones y el servidor valida cada jugada, simula el resultado y es la única fuente de verdad. Cada partida es un registro de eventos con número de secuencia en PostgreSQL, que se puede reproducir jugada por jugada y que sobrevive a reinicios y a publicaciones. Corre en Azure Container Apps con al menos dos réplicas coordinadas por Redis y PostgreSQL, lee sus secretos de Key Vault con identidad administrada y se despliega con Bicep y GitHub Actions, con aprobación manual para producción.

1.1 Enlaces

Qué	Dónde
Servidor en Azure (disponible durante la revisión)	https://ashenfall-server-prod.icytree-9cfd11e3.canadacentral.azurecontainerapps.io
Contrato REST (OpenAPI 3.1, publicado en /docs)	https://ashenfall-server-prod.icytree-9cfd11e3.canadacentral.azurecontainerapps.io/docs
Repositorio en GitHub	https://github.com/kevinmirandaa/ashenfall-game
Versión evaluada	Etiqueta **v1.0.0** del repositorio
Tablero de trabajo (Jira, proyecto SCRUM)	kevinmirandaa04.atlassian.net, proyecto SCRUM
Documentación en el repositorio	Carpeta `docs/`, con índice en `docs/README.md`

1.2 Dónde está cada requisito del enunciado

Parte del enunciado	Sección
Requisitos funcionales del servidor (usuarios, salas, emparejamiento, turnos, conexión, fin de partida, historial, administración)	4
Reglas del servidor (autoritativo, orden, idempotencia, vista por jugador, durabilidad, aislamiento, limpieza, reloj)	5 y 8
Lenguaje y stack, dominio puro	2 y 3
Comunicación y contratos (WSS, AsyncAPI 3.0, OpenAPI 3.1 en /docs, RFC 9457, validación, heartbeat y códigos de cierre)	6
Seguridad (autenticación, autorización, límites, transporte, secretos, cadena de suministro, modelo de amenazas)	7
Rendimiento y optimización (prueba de carga, capacidad y punto de quiebre, mensajes compactos, optimización medida)	9
Arquitectura y despliegue en Azure (topología, Bicep, identidad, costo)	3, 10 y 14
CI/CD, publicaciones sin cortar partidas y rollback	11
Observabilidad (OpenTelemetry, logs, métricas, sondas, alertas)	12
Pruebas y calidad	13
Entregables (repositorio, README, URL pública, etiqueta v1.0.0) y rúbrica	15 y 16

2. El servidor: qué se construyó y dónde está

2.1 Resumen

Aspecto	Detalle
Nombre y versión	ASHENFALL, versión 1.0.0 (etiqueta v1.0.0)
Lenguaje	TypeScript en modo estricto (strict: true, sin any) sobre Node.js 22 (el enunciado pide Node 20 o superior)
Framework HTTP	Fastify 5, con @fastify/helmet, @fastify/cors, @fastify/rate-limit, @fastify/swagger y @fastify/static
Tiempo real	WebSocket con la librería ws, sobre WSS; sin SignalR ni Socket.IO
Datos duraderos	PostgreSQL 16 (Flexible Server): log de eventos de cada partida, usuarios, salas, historial, tienda y auditoría
Estado compartido entre réplicas	Redis 7: difusión de eventos, presencia de los usuarios y contadores de límite de tasa
Contratos	OpenAPI 3.1 (contracts/openapi.json y /docs) y AsyncAPI 3.0 (contracts/asyncapi.json)
Dónde corre	Azure Container Apps, suscripción Azure for Students, región canadacentral, grupo de recursos rg-ashenfall-prod
Imagen	Docker multietapa sobre node:22-alpine, usuario no root, publicada en un Azure Container Registry privado
Infraestructura	Bicep (infra/main.bicep) y GitHub Actions con OIDC, sin secretos
Cliente de prueba	Phaser 3 con Vite, servido por el propio servidor desde el mismo origen (sin otro servicio que pagar y sin CORS)

2.2 Estructura del repositorio

Carpeta	Contenido
packages/domain	Motor de reglas puro: partida, física, turnos, arenas, vista por jugador y bot. Sin red, base de datos ni framework.
apps/server	El servidor: REST, WebSocket, PostgreSQL (migraciones SQL en migrations/), Redis y telemetría.
apps/bot	Bots reproducibles: load.ts (carga y punto de quiebre) y redeploy.ts (partidas abiertas durante una publicación).
apps/client	Cliente de prueba con Phaser.
contracts	openapi.json y asyncapi.json, generados desde el código; una prueba falla si dejan de coincidir.
infra	Bicep del ambiente completo y parámetros de dev y prod.
scripts	deploy.sh y rollback.sh de Azure, y la generación de arte y de esta documentación.
.github	Workflows de CI, despliegue, CodeQL, gitleaks y escaneo de la imagen, y la configuración de Dependabot.
docs	Documentación: decisiones, modelo de amenazas, reporte de carga, observabilidad, guía de Azure, costo y cumplimiento.

2.3 Módulos del servidor (apps/server/src)

Módulo	Responsabilidad
auth	Registro, login, refresh rotativo, logout, perfil; hasher Argon2id acotado; limitador de login por cuenta.
rooms	Salas públicas y privadas con código, unirse, salir, marcar listo; inicio con 2 o más listos.
matchmaking	Cola rápida: agrupa jugadores en partidas nuevas, con espera máxima configurable.
matches	MatchEngine (orden por partida), almacén de eventos, historial y replay, estadísticas, bots del servidor y sweeper de plazos.
realtime	Pasarela WebSocket: tickets de un solo uso, heartbeat, validación de mensajes, presencia, bus de eventos sobre Redis y códigos de cierre.
shop	Catálogo y compras con precios del servidor y billetera bloqueada.
admin	Partidas activas y conexiones, terminar una partida, suspender y reactivar usuarios, y auditoría.
security, db, telemetry, contracts	Cabeceras y errores 500 genéricos; pool de PostgreSQL con identidad de Entra; OpenTelemetry; esquemas de los contratos.

3. Arquitectura y patrones de diseño

3.1 Capas

El motor de reglas vive en un módulo de dominio puro (packages/domain) sin dependencias de red, base de datos ni framework, como exige el enunciado. El servidor conoce al dominio y el dominio no conoce a nadie; ESLint lo hace cumplir y no deja importar red, base de datos, reloj ni azar globales dentro del dominio. Por eso las reglas, la física y la vista por jugador se prueban sin ninguna dependencia externa.

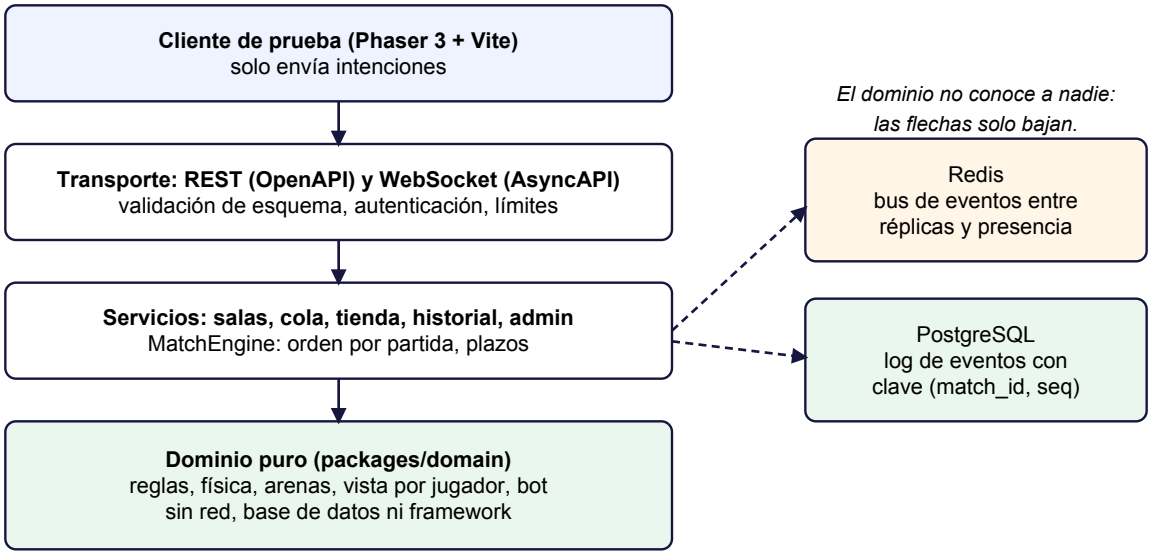


Figura 1. Capas del código: las dependencias solo bajan hacia el dominio puro.

3.2 Topología en Azure

Es la topología de referencia del enunciado. Los jugadores se conectan por HTTPS y WSS a cualquier réplica del servidor en Container Apps; el estado caliente y la difusión de eventos entre réplicas pasan por Redis; las jugadas confirmadas se guardan en PostgreSQL; los secretos se leen de Key Vault con identidad administrada. Un componente difiere de la referencia, con su justificación: Redis corre como una Container App interna en el mismo entorno en vez de Azure Cache for Redis, porque el costo de este último no cabe en el crédito de Azure for Students.

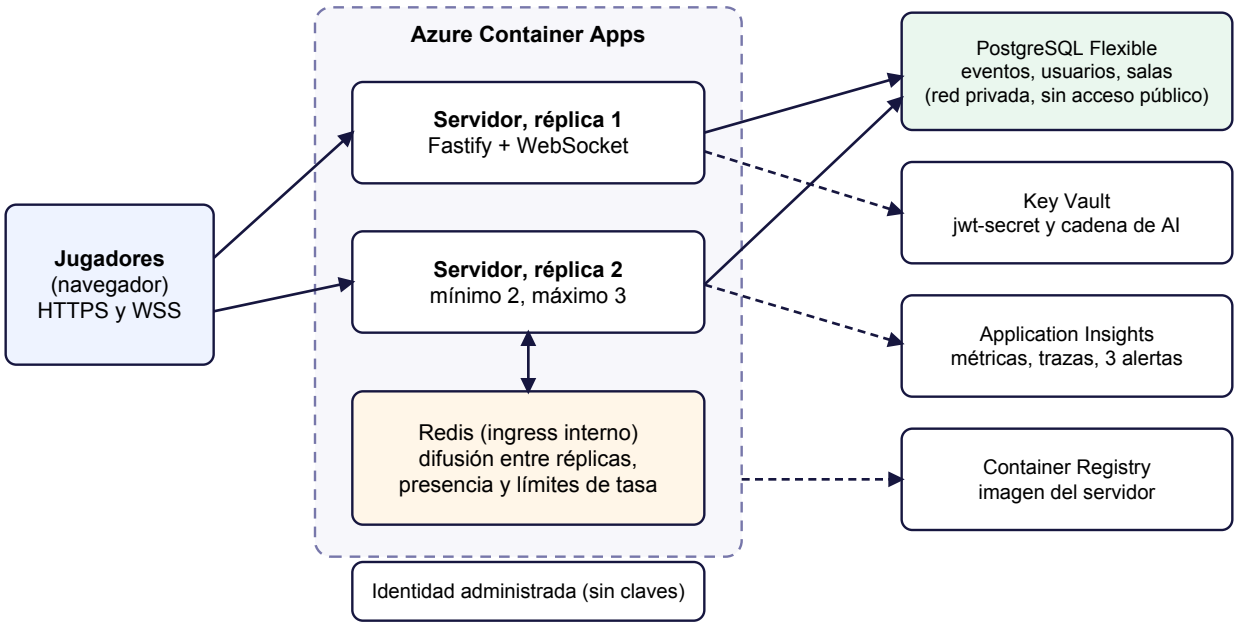


Figura 2. Topología de producción en Azure (región canadacentral).

3.3 Decisiones de diseño y su justificación

Decisión	Por qué
Event sourcing: cada partida es un log de eventos con seq	Da durabilidad, reproducción jugada por jugada y que cualquier réplica reconstruya el estado; un reinicio no pierde nada confirmado.
PostgreSQL para el log y Redis solo para lo efímero	Lo que no se puede perder va a la base transaccional; Redis (difusión, presencia, límites) puede perder datos sin consecuencia porque el cliente recupera el estado con sync.
Container Apps en vez de App Service	Revisiones con división de tráfico, drenado y rollback de tráfico nativos, y escala por solicitudes HTTP concurrentes.
JWT emitidos por el propio servidor	El enunciado lo permite; evita depender de un proveedor externo y permite tokens de 15 minutos con refresh rotativo.
Mensajes en JSON	Con los campos mínimos, unos 302 bytes por mensaje de media (sección 9.4); MessagePack no compensa la complejidad adicional.
Cliente servido por el mismo servidor	Mismo origen: sin CORS abierto y sin un segundo servicio que pagar.
Un solo Container Apps Environment	Azure for Students permite uno por suscripción (sección 10.7); prod es el único ambiente desplegado.

3.4 Patrones usados

Patrón	Dónde y para qué
Comando y decisión pura	Un comando (disparar, agotar tiempo, desconectar, reconectar) entra al dominio, que devuelve una decisión: aplicada con sus eventos, duplicada o rechazada con un código tipado.
Cola por partida y control optimista	Fila por partida en cada réplica y clave primaria (match_id, seq) entre réplicas, con hasta 5 reintentos al perder un conflicto.
Publicación y suscripción	Bus de eventos sobre Redis: cada réplica entrega a sus conexiones locales los eventos ya confirmados.
Proyección por jugador	toPlayerView y toPlayerEvent proyectan estado y eventos a lo que cada jugador puede ver.
Inyección de dependencias	El azar (RandomSource criptográficamente seguro), el reloj y la base de datos entran desde afuera, lo que permite probar con simulaciones.
Barrido de plazos sin líder	Todas las réplicas barren los plazos vencidos; si dos coinciden, el dominio rechaza el comando repetido y (match_id, seq) deja confirmar uno solo.

3.5 Flujo de una jugada

- El cliente envía `shoot` con actionId, expectedVersion, arma, ángulo y potencia.
- La pasarela valida el esquema, el tamaño (máximo 4 KiB) y la tasa de mensajes antes de que lleguen al dominio; un mensaje inválido se rechaza sin cerrar la partida.
- El motor encola la jugada en la fila de la partida, carga el log desde PostgreSQL y reproduce el estado.
- El dominio decide: ¿es su turno?, ¿la versión es vigente?, ¿el actionId ya se aplicó?
- Si se aplica, se insertan los eventos con clave (match_id, seq); si otra réplica ganó ese seq, se relee y se decide de nuevo (hasta 5 veces).
- Los eventos confirmados se publican por Redis; cada réplica los entrega con la vista de cada jugador y el tirador recibe su ack.

3.6 Ciclo de vida de una partida

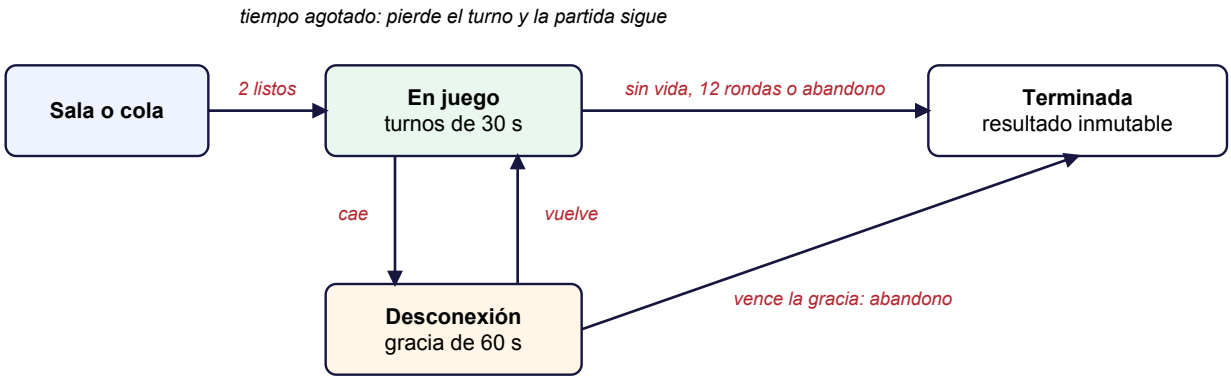


Figura 3. Ciclo de vida de una partida.

Los plazos de cada partida están en PostgreSQL (matches.next_deadline_ms) y todas las réplicas barren los vencidos, así que un reinicio no pierde temporizadores. Si un turno se agota, el jugador pierde el turno sin daño; dos turnos agotados seguidos cuentan como abandono. Al desconectarse, el jugador tiene 60 segundos para volver y recibe el estado actual; si no vuelve, pierde por abandono. El resultado se guarda una vez y no cambia.

3.7 Modelo de datos

Tabla	Contenido
users, refresh_tokens	Cuentas (con rol y estado de suspensión) y refresh tokens: solo se guarda el hash, con familia para detectar reutilización.
matches, match_players, match_events, match_results	La partida, sus participantes, el log de eventos con clave (match_id, seq) y el resultado inmutable.
rooms, room_members, matchmaking_queue	Salas con código de invitación, sus miembros y la cola rápida con expiración.
wallets, wallet_transactions, cosmetic_catalog, player_cosmetics	Monedas, movimientos y catálogo de personajes comprables (los nombres de las dos últimas son heredados del diseño inicial).
story_progress, admin_audit_logs	Progreso del modo historia y auditoría de las acciones de administración (autor, motivo y fecha).

Las migraciones son ocho scripts SQL numerados (migrations/001 a 008) que se aplican al arrancar el servidor.

4. Requisitos funcionales del servidor

Los nombres de endpoints y mensajes los define el equipo en sus contratos. La API REST tiene 30 rutas y 33 operaciones; el tiempo real usa un canal WebSocket (/ws) con 7 tipos de mensaje.

Capacidad	Qué hace	Implementación
Usuarios	Registro, inicio y cierre de sesión, renovación del token y perfil.	POST /v1/auth/register, login, refresh y logout; GET /v1/me. JWT de 15 minutos con refresh token rotativo de 30 días; contraseñas con Argon2id.
Salas	Crear partida pública o privada con código, listar, unirse, salir y marcar «listo».	POST y GET /v1/rooms, GET /v1/rooms/{id}, join, join-by-code, leave y ready. Inicia solo con 2 jugadores listos y la capacidad se respeta aunque varios se unan a la vez (prueba de concurrencia).
Emparejamiento	Cola rápida que agrupa jugadores en partidas nuevas.	POST, GET y DELETE /v1/matchmaking. Un jugador nunca queda en dos partidas; espera máxima de 60 s, configurable con MATCHMAKING_MAX_WAIT_MS.
Turnos	Enviar jugada, recibir el nuevo estado y saber a quién le toca.	Mensaje shoot por WebSocket. Solo juega quien tiene el turno; la jugada inválida recibe un error tipado (NOT_YOUR_TURN, STALE_VERSION, TURN_EXPIRED…); el temporizador de 30 s tiene como acción por defecto perder el turno.
Conexión	Detectar caídas, reconectar y recuperar el estado.	Heartbeat ping/pong cada 15 s, ticket de un solo uso, snapshot al reconectar y periodo de gracia de 60 s configurable (RECONNECT_GRACE_MS); al vencer, abandono.
Fin de partida	Declarar ganador o empate y registrar el resultado.	El resultado se guarda una vez y no cambia (match_results); GET /v1/matches/{id}/replay reproduce la partida jugada por jugada.

Capacidad	Qué hace	Implementación
Historial	Partidas jugadas y resultados por usuario.	GET /v1/me/matches, paginado; el replay solo lo ve un participante (404 idéntico al de un recurso inexistente).
Administración	Ver partidas activas y conexiones, terminar una partida, suspender un usuario.	GET /v1/admin/matches, POST terminate, suspend y unsuspend, GET /v1/admin/audit. Solo el rol administrador; cada acción queda auditada con autor, motivo y fecha.

5. Reglas del servidor

Regla del enunciado	Cómo se cumple
Servidor autoritativo	El cliente envía ángulo y potencia, nunca resultados. El dominio simula el tiro con una física determinista y versionada (PHYSICS_VERSION) y decide el daño.
Orden por partida y número de secuencia	Las jugadas de una partida se procesan una a una (fila por partida en cada réplica) y cada evento lleva un seq. Entre réplicas, la clave primaria (match_id, seq) deja confirmar a una sola; la otra relee el log y decide de nuevo. Una jugada sobre una versión vieja se rechaza (STALE_VERSION).
Idempotencia	Cada jugada lleva un actionId generado por el cliente y guardado en el evento. Si se reenvía, el servidor responde con un ack marcado duplicate y no la aplica dos veces; el mismo actionId usado por otro jugador se rechaza (ACTION_ID_CONFLICT).
Vista por jugador	Cada evento se proyecta a la vista de su destinatario. Ningún mensaje, endpoint ni log filtra el ángulo, la potencia ni el actionId del rival (prueba en mensajes, REST y logs).
Durabilidad	El estado se reconstruye siempre desde PostgreSQL; ninguna jugada confirmada vive solo en memoria. Si una réplica se reinicia o se publica una versión nueva, las partidas continúan.
Aislamiento	Un fallo en una partida no bloquea la fila de otra ni tumba el proceso: errores contenidos por mensaje y por partida, con una prueba de log dañado en una partida sin afectar a las demás.
Limpieza	Los plazos están en PostgreSQL y todas las réplicas barren los vencidos; las partidas inactivas terminan, las salas sin iniciar se cierran a la hora y la cola filtra por expiración.
Reloj y azar del servidor	Temporizadores y sorteos (terreno, obstáculos) se resuelven en el servidor con un generador criptográficamente seguro (crypto) que se inyecta al dominio.

6. Comunicación y contratos

6.1 API REST (OpenAPI 3.1)

Usuarios, salas, emparejamiento, historial, tienda y administración están en OpenAPI 3.1, publicada en /docs con Swagger UI y generada desde el código: una prueba falla si el contrato y las rutas dejan de coincidir. Las 30 rutas se reparten así:

Grupo	Rutas
Sondas	GET /health/live, GET /health/ready
Autenticación y perfil	POST /v1/auth/register, login, refresh, logout; GET /v1/me; POST /v1/me/character; POST /v1/ws-ticket
Salas y cola	POST y GET /v1/rooms; GET /v1/rooms/{id}; POST join, leave, ready y /v1/rooms/join-by-code; POST, GET y DELETE /v1/matchmaking
Partidas e historial	GET /v1/me/match, GET /v1/me/matches, GET /v1/matches/{id}/replay, GET /v1/me/stats, GET /v1/me/story, POST /v1/matches/bot
Tienda	GET /v1/shop/catalog, POST /v1/shop/purchases, GET /v1/me/inventory
Administración	GET /v1/admin/matches, POST /v1/admin/matches/{id}/terminate, POST /v1/admin/users/{id}/suspend y unsuspend, GET /v1/admin/audit

6.2 Errores: RFC 9457 Problem Details

Todos los errores de la API tienen el formato de Problem Details, con un código propio estable. Respuesta real del servidor a un registro sin correo:

```
{"type":"https://ashenfall.invalid/problems/validation-failed",
"title":"La petición no cumple el esquema","status":400,
"code":"VALIDATION_FAILED","instance":"/v1/auth/register",
"errors":[{"path":"","message":"must have required property 'email'"}]}
```

Los límites de tasa responden 429 con la cabecera Retry-After, y un error interno (500) devuelve un Problem Details genérico, sin detalles internos.

6.3 Protocolo en tiempo real (AsyncAPI 3.0)

Un solo canal WebSocket seguro (wss://) con mensajes versionados (campo v, hoy 1) documentados en AsyncAPI 3.0 con su esquema, ejemplos y errores posibles. No hay polling para saber si cambió el turno: el servidor empuja cada evento. El enunciado permite WebSocket directo; se usa la librería ws sin intermediarios.

Mensaje	Dirección	Contenido
shoot	cliente → servidor	matchId, actionId (UUID del cliente), expectedVersion, weaponId, angleDeg, power
sync	cliente → servidor	matchId; pide un snapshot si el cliente detecta un hueco en seq
welcome	servidor → cliente	userId, al autenticar el handshake

Mensaje	Dirección	Contenido
snapshot	servidor → cliente	Vista completa de la partida para ese jugador (al conectar, reconectar o sync)
event	servidor → cliente	seq, tiempo y un evento (match_started, shot_resolved, turn_started, turn_timed_out, player_disconnected, player_reconnected, match_finished)
ack	servidor → cliente	actionId, seq y duplicate (true si ya se había aplicado)
error	servidor → cliente	code tipado y, si aplica, el actionId

Ejemplos del formato de los mensajes (valores ilustrativos):

```
→ {"v":1,"type":"shoot","matchId":"…","actionId":"3b13d35a-…","expectedVersion":14,
"weaponId":"dagger","angleDeg":45,"power":60}
← {"v":1,"type":"ack","actionId":"3b13d35a-…","seq":16,"duplicate":false}
← {"v":1,"type":"error","code":"STALE_VERSION","actionId":"3b13d35a-…"}
```

Códigos de error tipados de WebSocket: INVALID_MESSAGE, RATE_LIMITED, INTERNAL_ERROR, MATCH_NOT_FOUND, MATCH_FINISHED, NOT_YOUR_TURN, STALE_VERSION, TURN_EXPIRED, INVALID_WEAPON, INVALID_ANGLE, INVALID_POWER y ACTION_ID_CONFLICT.

6.4 Validación de mensajes

Cada mensaje entrante se valida contra su esquema (tipos, rangos y tamaño máximo de 4 KiB, aplicado también por la propia librería ws con maxPayload) antes de llegar al dominio; los campos extra se rechazan. Un mensaje inválido recibe un error INVALID_MESSAGE y no cierra la conexión ni la partida. En REST, el cuerpo máximo es de 16 KiB y cada ruta valida su cuerpo con el esquema del contrato.

6.5 Heartbeat, cierre ordenado y códigos de cierre

El servidor envía ping cada 15 s y cierra la conexión si no recibe pong; en un apagado ordenado cierra las conexiones con el código 1001 y el cliente reconecta con un ticket nuevo. La presencia de cada usuario se renueva con cada heartbeat y caduca a los 45 s.

Código de cierre	Significado
1001	Cierre ordenado del servidor (publicación o reinicio): el cliente reconecta.
4003	Usuario suspendido por un administrador.
4008	Demasiadas conexiones para ese usuario (máximo 3).
4009	Se perdió el heartbeat.
4029	Demasiados mensajes por segundo: límite de tasa por conexión.

7. Seguridad

7.1 Autenticación

- **Tokens:** JWT firmados con HS256 y emitidos por el propio servidor (lo permite el enunciado), con emisor y audiencia verificados y el algoritmo fijado (un token con alg: none se rechaza). El token de acceso dura **15 minutos**.
- **Refresh token rotativo:** dura 30 días y solo se guarda su hash. Cada renovación entrega uno nuevo y reutilizar uno ya usado revoca toda la familia.
- **Contraseñas:** Argon2id con 19 MiB de memoria, 2 iteraciones y paralelismo 1 (configuración mínima recomendada por OWASP). El hasher admite una contraseña a la vez y una fila de 32; si se llena, el servidor responde 503 en vez de agotar CPU y memoria a las partidas en curso. El login hace una verificación señuelo si el correo no existe, para no revelar qué cuentas existen.
- **Secreto de firma:** mínimo 32 caracteres, leído de Key Vault; el servidor se niega a arrancar sin él y no tiene ningún valor por defecto.
- **WebSocket:** se autentica en el handshake con un **ticket de un solo uso** de 30 segundos (POST /v1/ws-ticket, guardado en Redis y consumido al conectar). Sin un ticket válido, la conexión se cierra.

7.2 Autorización

- Solo los participantes actúan en su partida: las salas, partidas y replays ajenos responden 404, idéntico al de un recurso inexistente, para que no se pueda sondear.
- Solo juega quien tiene el turno: el dominio valida turno y versión en cada mensaje shoot.
- Solo el administrador usa /v1/admin: el rol se lee de la base de datos en cada petición y ninguna ruta de la API lo concede (se asigna con una herramienta de línea de comandos); cada acción de administración queda auditada.
- Un usuario suspendido pierde sus conexiones en todas las réplicas (código 4003).

7.3 Límites

Límite	Valor	Configuración
Tasa global de HTTP	600 peticiones por minuto, por usuario (token válido) o por IP	RATE_LIMIT_GLOBAL_PER_MINUTE
Tasa de rutas de autenticación	20 por minuto por IP	RATE_LIMIT_AUTH_PER_MINUTE
Intentos de login por cuenta	10 por minuto, en cualquier IP	RATE_LIMIT_LOGIN_PER_MINUTE
Respuesta al exceder	HTTP 429 con Retry-After	ProblemError RATE_LIMITED
Mensajes por conexión WebSocket	20 por segundo; al triplicarlo se cierra con 4029	Pasarela
Tamaño máximo de mensaje	4 KiB en WebSocket y 16 KiB en el cuerpo HTTP	maxPayload y bodyLimit
Conexiones por usuario	3, contadas entre todas las réplicas con Redis	Pasarela y presencia
Sonda /health/ready	120 por minuto	Para no saturar la base con las sondas

Los contadores viven en Redis, así que el límite vale entre réplicas; si Redis cae, el límite de login sigue por réplica.

7.4 Transporte

- **Solo HTTPS y WSS:** el ingress de Container Apps tiene allowInsecure en false y redirige `http://` con 301; el servidor corre con ENFORCE_HTTPS=1 (la CSP solo permite wss:) y con TRUST_PROXY=1 para ver la IP real tras el ingress.
- **Cabeceras de seguridad** con @fastify/helmet: Strict-Transport-Security (max-age de un año, includeSubDomains), Content-Security-Policy (default-src, script-src y connect-src solo del mismo origen, object-src none, frame-ancestors del mismo origen, upgrade-insecure-requests), X-Content-Type-Options nosniff y las demás de helmet.
- **CORS restringido:** ALLOWED_ORIGINS está vacío en producción (el cliente se sirve del mismo origen), así que ningún origen ajeno recibe Access-Control-Allow-Origin; métodos GET, POST y DELETE y cabeceras authorization y content-type.
- **Origen del WebSocket restringido:** un handshake con un Origin ajeno recibe 403.

7.5 Secretos e identidad administrada

- El secreto de los JWT (jwt-secret) y la cadena de Application Insights están en Key Vault; el Container App los lee como referencias de secreto con la identidad administrada id-ashenfall-prod, que tiene solo los roles Key Vault Secrets User (sobre el Key Vault) y AcrPull (sobre el registro).
- PostgreSQL se accede con un token de Microsoft Entra de esa misma identidad: la autenticación por contraseña está desactivada y la cadena de conexión no lleva clave. Redis no tiene contraseña porque solo tiene ingress interno y vive dentro del entorno.
- Ninguna clave ni cadena de conexión está en la configuración en texto plano (se comprobó con az containerapp show: solo las referencias a Key Vault) ni en el repositorio: gitleaks revisa el historial completo en cada cambio.
- El pipeline entra a Azure con OIDC federado (sin secretos): una identidad administrada ashenfall-github con credenciales federadas para los environments dev y prod de GitHub.

7.6 Cadena de suministro

Control	Configuración
Dependabot	Revisa cada semana las dependencias npm y las acciones de GitHub y abre pull requests.
CodeQL (análisis de código)	En cada pull request, en cada push a main y cada lunes.
gitleaks (secretos)	En cada pull request y push; revisa todo el historial con --exit-code 1.
Trivy (imagen del contenedor)	En cada pull request, push y cada lunes; falla si hay vulnerabilidades CRITICAL.
Imagen del contenedor	Multietapa; se ejecuta como el usuario no root «node»; se eliminan npm y corepack de la imagen final para quitar dependencias vulnerables de la base; HEALTHCHECK a /health/live.

7.7 Modelo de amenazas

El modelo completo (activos, fronteras de confianza, amenazas por STRIDE, defensa y prueba) está en docs/threat-model.md. Resumen:

Amenaza	Defensa	Prueba
Fuerza bruta de contraseñas	Límite por IP y por cuenta; Argon2id; verificación señuelo	rate-limit.test.ts, auth.test.ts
Robo o reutilización de refresh token	Rotación; reutilizar uno revoca la familia; solo se guarda el hash	auth.test.ts
JWT falsificado	Algoritmo fijo; emisor y audiencia verificados	auth.test.ts
Secuestro del WebSocket desde otra web	Ticket de un solo uso de 30 s y orígenes restringidos	gateway.test.ts
Enviar daño o resultados	El esquema rechaza campos extra; el servidor calcula todo	gateway.test.ts

Amenaza	Defensa	Prueba
Jugar fuera de turno o sobre estado viejo	El dominio valida turno y versión	match.test.ts, gateway.test.ts
Reenviar una jugada	actionId idempotente y clave (match_id, seq)	gateway.test.ts, resilience.test.ts
Ver datos del rival o partidas ajenas	Vista por jugador; 404 idéntico al de un recurso inexistente	view.test.ts, history.test.ts
Duplicar monedas o fijar un precio	Precio del servidor, billetera bloqueada y claves únicas	shop.test.ts, event-store.test.ts
Inundar la API o el WebSocket	Límites por usuario e IP, tamaño máximo, tasa por conexión y conexiones por usuario	rate-limit.test.ts, gateway.test.ts
Pico de logins que agota CPU o memoria	Argon2id acotado con fila y respuesta 503	password-hasher.test.ts
Usar funciones de administrador	Rol leído de la base en cada petición; ninguna ruta lo concede; auditoría	admin.test.ts
Secreto en el repositorio o en logs	gitleaks en CI; sin valores por defecto; logs sin cabeceras ni cuerpos	CI, hardening.test.ts
Imagen o dependencias comprometidas	Usuario no root; Trivy, CodeQL y Dependabot en CI	CI

7.8 No filtración de información oculta

El juego casi no tiene información oculta: la posición del rival y la trayectoria son públicas dentro de la partida por decisión de diseño (la cámara las muestra a ambos), y el enunciado marca la información oculta como recomendada, no obligatoria. Aun así hay una vista por jugador: el rival nunca recibe el ángulo, la potencia ni el actionId de una jugada, ni por WebSocket, ni por REST (el replay), ni en los logs, y hay pruebas que lo verifican en cada canal.

7.9 Riesgos aceptados

- Un cliente modificado puede calcular el tiro perfecto, porque la posición y la física son públicas por diseño.
- Agotar el límite de login por cuenta bloquea a su dueño un minuto; se prefiere a permitir fuerza bruta distribuida.
- Con Redis caído, los límites por plugin dejan de aplicarse para no tumbar el servicio y no se pueden emitir tickets nuevos.
- El ticket viaja en la URL del handshake: es de un solo uso y dura 30 s, y el servidor no registra esa URL.
- Una réplica que cae de golpe no registra desconexiones; sus partidas terminan por los plazos de turno.

7.10 Verificado en Azure (7 de noviembre de 2026)

- `http://` responde 301; hay HSTS y CSP; un Origin ajeno en /ws recibe 403 y no recibe cabecera CORS.
- El login devuelve 429 con Retry-After tras repetir intentos fallidos.
- Los secretos del Container App son referencias a Key Vault con identidad administrada; la cadena de PostgreSQL no lleva contraseña.
- PostgreSQL tiene el acceso público desactivado y vive en una subred delegada; Redis solo tiene ingress interno.
- El environment prod de GitHub tiene un revisor obligatorio y el despliegue del 7 de noviembre esperó su aprobación.

8. Sesiones y concurrencia

Problema	Solución en el servidor
Muchas partidas simultáneas sin interferir	Cada partida tiene su propia fila de comandos y su propio log; un fallo se contiene por partida. Se midieron 800 partidas simultáneas en una réplica sin degradación (sección 9).
Jugadas simultáneas	Una sola se confirma por (match_id, seq); la otra se relee y se decide contra el estado nuevo, por lo que llega como rechazo tipado si ya no procede.
Mensajes duplicados	actionId idempotente: el reenvío devuelve ack con duplicate en true.
Mensajes fuera de orden o sobre estado viejo	expectedVersion frente a la versión del estado: STALE_VERSION; el cliente pide sync.
Varias instancias atendiendo la misma partida	Cualquier réplica reconstruye el estado desde PostgreSQL; el bus de Redis reparte los eventos y, si un mensaje se pierde, el cliente detecta el hueco en seq.
Temporizadores	El plazo de cada partida está en PostgreSQL; todas las réplicas barren los vencidos sin líder; el bot del servidor decide también por ese barrido.
Desconexión y reconexión	Presencia por usuario en Redis; al reconectar, el servidor envía un snapshot. La desconexión solo cuenta si el usuario no tiene ninguna conexión en ninguna réplica, y un cierre tardío no pisa a la conexión nueva.
Abandono	Al vencer la gracia de 60 s, o con dos turnos agotados seguidos, la partida termina por abandono.
Reinicios y publicaciones	Ninguna jugada confirmada vive solo en memoria; las revisiones nuevas reciben el tráfico solo cuando pasan su prueba de salud, y las conexiones se cierran de forma ordenada (1001).
Limpieza	Partidas inactivas expiran por los plazos; salas sin iniciar se cierran a la hora; la cola filtra por expiración; el barrido de limpieza corre cada 30 s.

La resiliencia se probó en tres niveles: pruebas automáticas de concurrencia, reinicio y aislamiento (resilience.test.ts, gateway.test.ts, event-store.test.ts); un bot que reinicia un servidor local a media partida (6 reconexiones y 98 de 98 jugadas confirmadas y coincidentes con el replay); y las corridas en Azure de la sección 9.5, incluida una publicación de revisión con partidas abiertas.

9. Rendimiento y optimización

La prueba de carga es reproducible: el bot de apps/bot juega partidas reales por HTTP y WebSocket y mide el tiempo desde que se envía una jugada hasta que ambos jugadores reciben el resultado. Las metas del enunciado son p95 menor a 200 ms, ninguna jugada perdida o duplicada y error menor a 1 %. Las pruebas locales se hicieron con una réplica limitada a 1 CPU y 1 GB, con PostgreSQL y Redis sin límite y el bot en la misma máquina (5 de noviembre de 2026). Los comandos para reproducirlas están en docs/load-report.md.

9.1 Ritmo normal: un disparo por segundo y partida

Sin jugadas perdidas ni duplicadas y sin errores en ningún escalón: se cumplen las tres metas.

Partidas	Conexiones	Jugadas	Jugadas/s	p50	p95	p99	CPU réplica	Memoria máx.
25	50	300	23	9 ms	15 ms	16 ms	11 %	53 MB
50	100	600	46	7 ms	13 ms	17 ms	13 %	55 MB
100	200	1 200	92	4 ms	14 ms	17 ms	13 %	56 MB
200	400	2 400	184	3 ms	9 ms	24 ms	15 %	61 MB
400	800	4 800	369	2 ms	5 ms	7 ms	21 %	66 MB
800	1 600	9 600	737	2 ms	7 ms	17 ms	33 %	75 MB

9.2 Capacidad: CPU y memoria por partida y punto de quiebre

Entre 400 y 800 partidas, el consumo es de unos **0,03 puntos de CPU y 0,02 MB por partida activa** (la memoria incluye lo que el proceso ya tenía reservado, así que es una aproximación). Para el punto de quiebre se usaron diez disparos por segundo y partida, para saturar sin crear miles de cuentas; ninguna jugada se perdió ni duplicó, tampoco al degradarse.

Partidas	Jugadas/s	p50	p95	p99	CPU réplica	CPU PostgreSQL	Meta p95
200	1 692	7 ms	43 ms	52 ms	51 %	46 %	Cumple
400	1 781	119 ms	198 ms	225 ms	66 %	95 %	En el límite
800	1 845	327 ms	428 ms	453 ms	98 %	80 %	No cumple

Una réplica deja de cumplir la meta de latencia alrededor de 1 800 jugadas por segundo, y el rendimiento apenas sube a partir de ahí. A un disparo por segundo y partida se midió sin degradación hasta 800 partidas por réplica; extrapolando, el tope rondaría 1 700 partidas, pero esa cifra no se midió.

9.3 Optimizaciones medidas (antes y después)

Medida	Antes	Después
p95 de las jugadas con 100 partidas y 100 clientes enviando logins fallidos sin parar (Argon2id acotado a una contraseña a la vez)	202 ms	109 ms
Preparar 100 partidas con sus 200 cuentas (Argon2id con parámetros mínimos de OWASP)	57 s	5 s
Idas y vueltas a PostgreSQL por jugada	7	5
Jugadas por segundo en saturación (400 y 800 partidas, dos ejecuciones)	1 593 a 1 647	1 754 a 1 839
Listado de salas públicas (50 salas)	101 consultas, p95 7,5 ms	2 consultas, p95 1,1 ms
Consultas a PostgreSQL en reposo	4,42 transacciones/s	2,76 transacciones/s
Imagen Docker (disco)	565 MB	387 MB
Descarga de un jugador nuevo hasta empezar una partida	105 peticiones, 3,84 MB	47 peticiones, 1,29 MB

Coste aceptado de la primera optimización: una sola réplica procesa unos 20 logins por segundo en un pico, así que en un ataque algunos usuarios reciben 503 y deben reintentar; se prefiere a poner en riesgo las partidas en curso. El detalle de cada cambio está en docs/optimizacion.md.

9.4 Mensajes compactos

El WebSocket usa JSON con los campos mínimos y trayectorias en enteros muestreados: unos **302 bytes por mensaje** de media. El estado completo solo viaja al conectar o al pedir sync; el resto del tiempo se envían solo los eventos nuevos, proyectados a lo que cada jugador puede ver.

9.5 Mediciones en Azure (7 de noviembre de 2026)

Contra la URL pública de producción, con el bot en un portátil en Costa Rica, por lo que las latencias de extremo a extremo incluyen el viaje por internet hasta Canadá.

Prueba	Partidas en paralelo	Duración	Jugadas confirmadas	Partidas contrastadas	Perdidas / duplicadas / errores
Control, 1 réplica	6	11,2 min	473	18	0 / 0 / 0
Durante la publicación de la revisión con 2 réplicas	6	6,9 min	288	12	0 / 0 / 0
Sobre las 2 réplicas nuevas	8	7,1 min	518	16	0 / 0 / 0

- Cada partida se contrastó con su replay del servidor: toda jugada confirmada figura una sola vez.
- Las jugadas se repartieron entre las réplicas: 261 y 257 en las dos nuevas y 240 en la anterior, con partidas cuyos jugadores quedaban en réplicas distintas.

- La métrica del servidor ashenfall.shot.latency, leída en Application Insights, dio media de 15,6 ms en una primera carga corta (91 jugadas, máximo de 61 ms) y de 17,7 y 17,9 ms en las dos réplicas nuevas; el máximo en estas últimas fue de 203 ms en una jugada.
- De extremo a extremo con internet de por medio, una carga corta dio p50 de 141 ms y p95 de 216 ms: el p95 supera 200 ms por el viaje desde Costa Rica, no por el servidor.
- No se midió el punto de quiebre dentro de Azure; el de una réplica está medido en local.

10. Despliegue en Azure

10.1 Recursos desplegados

Todo está en el grupo de recursos **rg-ashenfall-prod**, región **canadacentral**, suscripción Azure for Students. Los nombres con sufijo (3fiowy) los genera el Bicep a partir del grupo de recursos para que sean únicos.

Recurso (nombre real)	Tipo	Configuración
ashenfall-server-prod	Container App	El servidor: 0,5 vCPU y 1 GiB, de 2 a 3 réplicas, revisiones múltiples, ingress externo HTTPS en el puerto 8080
ashenfall-redis-prod	Container App	Redis 7: 0,25 vCPU y 0,5 GiB, 1 réplica fija, ingress interno
cae-ashenfall-prod	Container Apps Environment	Entorno compartido, dentro de la subred snet-apps; registros enviados a Log Analytics
psql-ashenfall-prod-3fiowy	PostgreSQL Flexible Server	Versión 16, Standard_B1ms (Burstable), 32 GB sin crecimiento automático, backup de 7 días, sin alta disponibilidad, autenticación solo con Entra, acceso público desactivado
vnet-ashenfall-prod	Red virtual	Subredes snet-apps (Container Apps) y snet-postgres (delegada a PostgreSQL), con zona DNS privada ashenfall-prod.private.postgres.database.azure.com
kv-ash-prod-3fiowy	Key Vault	SKU standard, autorización por roles (RBAC); contiene jwt-secret y la cadena de Application Insights
acrashenfallprod3fiowy	Container Registry	SKU Basic; la imagen se etiqueta con los primeros 12 caracteres del sha del commit
id-ashenfall-prod	Identidad administrada (usuario)	Roles AcrPull sobre el registro y Key Vault Secrets User sobre el Key Vault; administrador de Entra en PostgreSQL
log-ashenfall-prod	Log Analytics	SKU PerGB2018, retención de 30 días, tope de 1 GB al día
appi-ashenfall-prod	Application Insights	Basado en el espacio de trabajo; recibe las métricas y trazas de OpenTelemetry
alert-restarts-prod, alert-postgres-cpu-prod, alert-latency-prod	Reglas de alerta	Tres alertas (sección 12.3)

10.2 Configuración del servidor en Container Apps

Parámetro	Valor
Recursos por réplica	0,5 vCPU y 1 GiB de memoria; NODE_OPTIONS=--max-old-space-size=512 para acotar el heap de Node
Réplicas	Mínimo 2 y máximo 3; regla de escala HTTP con 50 solicitudes concurrentes, sondeo cada 30 s y enfriamiento de 300 s
Revisiones	Modo Multiple: la revisión nueva nace sin tráfico, se prueba y luego recibe el 100 %
Sonda de vida	GET /health/live cada 10 s, falla tras 3 intentos
Sonda de disponibilidad	GET /health/ready cada 5 s, falla tras 3 intentos; comprueba PostgreSQL y Redis
Apagado ordenado	terminationGracePeriodSeconds de 120 s para cerrar conexiones con el código 1001
Ingress	Externo, puerto 8080, allowInsecure en false (HTTP redirige con 301), WebSocket habilitado

10.3 Variables de entorno

Variable	Valor o procedencia
NODE_ENV, OTEL_SERVICE_NAME	production; ashenfall-server
DATABASE_URL	postgres://id-ashenfall-prod@psql-ashenfall-prod-3fiowy.postgres.database.azure.com:5432/ashenfall?sslmode=require (sin contraseña: se usa un token de Entra)
AZURE_CLIENT_ID	Identificador de la identidad administrada con la que se pide el token de Entra
REDIS_URL	redis://ashenfall-redis-prod:6379 (nombre interno del Container App)
JWT_SECRET	Referencia al secreto jwt-secret de Key Vault
APPLICATIONINSIGHTS_CONNECTION_STRING	Referencia al secreto appinsights-connection-string de Key Vault
TRUST_PROXY, ENFORCE_HTTPS	1 y 1
ALLOWED_ORIGINS	Vacío: solo mismo origen
RATE_LIMIT_*, MATCHMAKING_MAX_WAIT_MS, RECONNECT_GRACE_MS	Valores por defecto de la sección 7.3 y de 60 s

10.4 Red

El servidor es la única pieza con ingress externo. Redis solo es accesible desde dentro del entorno por su nombre interno. PostgreSQL no tiene acceso público: solo se llega desde la red virtual, mediante la subred delegada y la zona DNS privada; la identidad administrada se autentica con Microsoft Entra.

10.5 Identidad y roles

Identidad	Para qué	Roles
id-ashenfall-prod (administrada, de usuario)	Acceso del servidor a los demás servicios, sin claves	AcrPull sobre el registro; Key Vault Secrets User sobre el Key Vault; administrador de Entra en PostgreSQL
ashenfall-github (administrada, de usuario)	Despliegue desde GitHub Actions con OIDC, sin secretos	Contributor y Role Based Access Control Administrator sobre la suscripción; credenciales federadas para los environments dev y prod de GitHub

10.6 Infraestructura como código

Todos los recursos están en Bicep (infra/main.bicep, con infra/dev.bicepparam y infra/prod.bicepparam) y un ambiente nuevo se levanta con una sola ejecución del pipeline: crea el grupo de recursos, la infraestructura base, construye y publica la imagen, crea el secreto y despliega la aplicación. Los requisitos previos (la identidad de GitHub con sus credenciales federadas, los roles en la suscripción y el registro de proveedores de recursos) se hacen una sola vez desde el portal y están en docs/azure-setup.md. El 7 de noviembre de 2026 el pipeline creó dev y prod desde grupos de recursos vacíos.

10.7 Límite de la suscripción y decisión

Azure for Students permite un solo entorno de Container Apps por suscripción (el despliegue falla con el error MaxNumberOfGlobalEnvironmentsInSubExceeded; se comprobó incluso con dev en otra región). Por eso dev y prod no pueden existir a la vez. Dev se desplegó desde cero con el pipeline, se probó y se borró para dejar sitio a prod; el despliegue automático a dev (variable AUTO_DEPLOY) queda apagado. El Bicep y el pipeline de dev siguen en el repositorio y funcionan en una suscripción sin ese límite, con dev automático tras cada CI verde y prod con aprobación manual. Con dev borrado, el pipeline admite lanzar solo prod (casilla solo_prod).

11. Integración y entrega continuas

11.1 Pipeline de integración

En cada pull request y en cada push a main, GitHub Actions ejecuta, con PostgreSQL 16 y Redis 7 reales como servicios:

- **build** de dominio, servidor, bot y cliente; **typecheck** con tsc en modo estricto; **linter** (ESLint con typescript-eslint estricto y las reglas de pureza del dominio);
- **pruebas** (294 pruebas), que incluyen la **validación de contratos**: OpenAPI y AsyncAPI deben coincidir con el código;
- **escaneo de seguridad**: CodeQL (código), gitleaks (secretos), Trivy (imagen del contenedor) y Dependabot (dependencias), como flujos propios;
- la imagen se construye en cada cambio para escanearla, y se publica en el registro de Azure en el despliegue.

11.2 Pipeline de despliegue (deploy.yml y scripts/deploy.sh)

- **Disparo:** tras un CI verde en main (el despliegue automático a dev se controla con AUTO_DEPLOY) o a mano con workflow_dispatch.
- **Dev** se despliega automáticamente; **prod** usa el environment prod de GitHub, que exige un revisor: el trabajo espera su aprobación antes de empezar.
- **Inicio de sesión:** azure/login con OIDC, sin secretos guardados.
- **Pasos de deploy.sh:** crear el grupo de recursos; desplegar la infraestructura base (registro, Key Vault, PostgreSQL, Redis); construir y publicar la imagen en el registro con el sha como etiqueta; crear jwt-secret en Key Vault si no existe (el valor nunca se imprime); desplegar la aplicación; hacer la prueba de humo; trasladar el tráfico; desactivar revisiones antiguas.

11.3 Publicaciones sin cortar partidas

- La revisión nueva nace con el **0 % del tráfico** y se prueba por su propia URL (/health/ready) antes de recibir nada.
- Solo entonces recibe el **100 %**; la revisión anterior queda activa como destino de rollback y las más viejas se desactivan (**drenado**).
- Al desactivar una revisión, sus réplicas reciben la señal de apagado y disponen de 120 s para cerrar las conexiones de forma ordenada (código 1001); los clientes reconectan con un ticket nuevo y reciben el estado actual, y como el estado está en PostgreSQL ninguna partida se pierde.
- **Evidencia:** el 7 de noviembre de 2026 se publicó una revisión nueva de prod con el pipeline mientras 6 partidas del bot seguían en juego: 0 jugadas perdidas, 0 duplicadas, 0 errores, con las partidas contrastadas con su replay (sección 9.5).

11.4 Rollback

scripts/rollback.sh pasa el 100 % del tráfico a la revisión anterior con un solo cambio de tráfico, sin construir nada. Lo mismo se puede hacer desde el portal (Gestión de revisiones). Se **probó en producción** el 7 de noviembre de 2026: tardó **22 segundos** y /health/ready respondió 200 durante el cambio; la meta del enunciado es menos de 5 minutos. Si la causa es un esquema de base de datos incompatible, el rollback de tráfico no revierte migraciones.

12. Observabilidad

12.1 OpenTelemetry hacia Application Insights

El servidor usa el SDK de OpenTelemetry con los exportadores de Azure Monitor y la cadena de conexión de Application Insights (leída de Key Vault). Las métricas se exportan cada 60 s y las trazas se muestrean al 10 % para controlar el costo; el cliente de telemetría es opcional en local y en las pruebas.

12.2 Métricas propias, trazas y logs

Elemento	Detalle
Partidas activas	ashenfall.matches.active: partidas en curso en todas las réplicas
Conexiones	ashenfall.connections: conexiones WebSocket en cada réplica
Jugadas por segundo	ashenfall.shots: contador de jugadas confirmadas; la tasa por segundo se calcula al consultar
Latencia por jugada	ashenfall.shot.latency: histograma en ms, desde que llega la jugada hasta repartirla, con límites alrededor de la meta de 200 ms
Trazas	Un span por comando (match.shoot, match.turn_timeout, match.player_disconnected…) con los atributos ashenfall.match_id y ashenfall.player_id; se comprobó en Application Insights
Logs estructurados	JSON con el id de la partida y del jugador y el id de traza para cruzar logs y trazas; sin cuerpos ni cabeceras de autorización; las sondas y los estáticos no se registran por petición

12.3 Sondas y alertas

Alerta o sonda	Condición
/health/live	El proceso responde (sonda de vida del Container App y HEALTHCHECK de la imagen)
/health/ready	PostgreSQL y Redis disponibles (sonda de disponibilidad); limitada a 120 por minuto
alert-restarts-prod (severidad 1)	Reinicios del contenedor mayores que 0 (suma), cada 5 min sobre 15 min
alert-postgres-cpu-prod (severidad 2)	CPU de PostgreSQL mayor que 80 % (promedio), cada 5 min sobre 15 min
alert-latency-prod (severidad 2)	Consulta sobre ashenfall.shot.latency: promedio mayor que 200 ms en 15 min

Las tres alertas están activas en Azure. Si se pasa el parámetro alertEmail al Bicep se les asocia un grupo de acción por correo; hoy solo se ven en el portal. Que una alerta dispare no se ha visto.

13. Pruebas y calidad

Hay 294 pruebas en 24 archivos (resultado del 7 de noviembre de 2026), que CI ejecuta en cada pull request junto con el linter y la revisión de tipos. Las pruebas de integración usan PostgreSQL y Redis reales.

Tipo de prueba	Qué cubre	Archivos
Dominio y física	Reglas, turnos, daño, terreno y obstáculos sorteados con su solución, vista por jugador, progresión y bot	match, physics, view, progression, bot
API	Usuarios, salas, historial, tienda, administración y contratos sincronizados con el código	auth, rooms, history, shop, admin, contracts
WebSocket y concurrencia	Orden, idempotencia, jugadas simultáneas, varias instancias, reconexión, abandono y sync	gateway, event-store, matchmaking
Resiliencia	Reinicio de una réplica, log dañado en una partida sin afectar a las demás, limpieza	resilience, pool, health
Seguridad	Autorización por partida y por rol, límites de tasa, cabeceras, logs sin secretos, no filtración al rival	hardening, rate-limit, password-hasher
Compras y recompensas	Compras concurrentes y recompensa duplicada	shop, event-store
Observabilidad y estáticos	Métricas, trazas y servicio de ficheros precomprimidos	telemetry, static, config

Además hay dos bots reproducibles: `apps/bot/src/load.ts` (carga y punto de quiebre) y `apps/bot/src/redeploy.ts` (partidas abiertas con clientes que se reconectan y reenvían con el mismo actionId, contrastadas con el replay).

14. Costo estimado en Azure

Es una estimación con precios de lista consultados el 5 de noviembre de 2026 (región northeurope, USD), no una factura; los precios cambian por región. Supuestos: mes de 30 días, sin descontar cuotas gratuitas y sin tráfico de salida. Detalle y supuestos en docs/azure-cost.md.

Recurso	Configuración	USD por mes
PostgreSQL B1ms (cómputo)	Burstable, sin alta disponibilidad	13,14
Container Registry Basic	SKU mínimo	5,00
Redis (Container App)	0,25 vCPU y 0,5 GiB, 1 réplica fija	5,8 a 19,4
Servidor de prod	0,5 vCPU y 1 GiB, 2 réplicas mínimas	23,4 a 77,8 (11,7 a 38,9 por réplica)
Log Analytics, Key Vault y alertas	Retención de 30 días, tope de 1 GB al día	desde 0; peor caso 69 en logs
Total aproximado de prod	Sin almacenamiento de PostgreSQL ni logs	**~47 a ~115**

Azure for Students suele dar 100 USD de crédito (confirmarlo en el portal), así que el costo es ajustado. Medidas aplicadas: contenedores de tamaño mínimo, heap de Node acotado, sin copiar métricas de plataforma a Log Analytics, sin registrar las sondas ni los estáticos, y trazas muestreadas al 10 %. Los recursos de dev se apagan o escalan a cero cuando no se usan (el servidor de dev tiene mínimo de 0 réplicas, y dev se borra tras probar). Pasado el periodo de revisión se recomienda borrar el grupo de recursos de producción para no gastar crédito.

15. Cómo correr y probar el servidor

15.1 En Azure

El servidor está disponible en https://ashenfall-server-prod.icytree-9cfd11e3.canadacentral.azurecontainerapps.io; la documentación de la API en https://ashenfall-server-prod.icytree-9cfd11e3.canadacentral.azurecontainerapps.io/docs y las sondas en /health/live y /health/ready.

15.2 En local (menos de 10 minutos)

Solo se necesita Docker Desktop abierto; no hace falta Node ni configurar secretos (con ASHENFALL_DEV_SECRETS=1 el compose genera una clave JWT aleatoria al arrancar).

```
git clone https://github.com/kevinmirandaa/ashenfall-game.git
cd ashenfall-game
docker compose up --build
```

Levanta PostgreSQL, Redis y el servidor; las migraciones se aplican al arrancar. El servidor queda en http://localhost:8080, la API en /docs y las sondas en /health/live y /health/ready. La imagen se construye sin caché en alrededor de un minuto en un Mac con Docker Desktop (medido el 7 de noviembre; depende de la red). Los puertos solo se publican en 127.0.0.1.

15.3 Pruebas y carga

```
npm ci
docker compose up -d --wait postgres redis
npm test

npm run build
node apps/bot/dist/load.js --steps 25,50,100 --think-ms 1000 --shots 12
```

Otros comandos: npm run lint, npm run typecheck y npm run test:coverage. La prueba de carga completa y el punto de quiebre están explicados en docs/load-report.md.

15.4 Administración

Ninguna ruta de la API concede el rol de administrador. Para dárselo a una cuenta ya registrada:

```
docker compose exec server node apps/server/dist/db/grant-admin-cli.js
correo@ejemplo.com
```

16. Cumplimiento del enunciado

La matriz completa, con 94 requisitos, su evidencia (código, prueba, documento, workflow o Bicep) y su estado, está en docs/cumplimiento-enunciado.md. Resultado: **92 requisitos cumplidos, 2 cumplidos parcialmente y ninguno incumplido**. La tabla siguiente sigue el orden del enunciado.

16.1 Requisitos técnicos y de despliegue

Requisito del enunciado	Cómo se cumple	Estado	Sección
Lenguaje con tipado estático y modo estricto	TypeScript strict: true, sin any (ESLint), Node 22	Cumple	2
Motor de reglas en un módulo de dominio puro	packages/domain, sin red, base de datos ni framework (regla de ESLint)	Cumple	3.1
WebSocket seguro (wss://), sin polling del turno	ws sobre WSS; el servidor empuja cada evento	Cumple	6.3
Protocolo versionado documentado con AsyncAPI 3.0	contracts/asyncapi.json: 7 mensajes con esquema, ejemplos y errores	Cumple	6.3
API REST con OpenAPI 3.1 publicada en /docs	30 rutas; Swagger UI en /docs	Cumple	6.1
Errores RFC 9457 Problem Details	Todos los errores, con código propio	Cumple	6.2
Cada mensaje validado antes del dominio; inválido no cierra la partida	Esquemas con tipos, rangos y 4 KiB; error INVALID_MESSAGE	Cumple	6.4
Heartbeat y cierre ordenado con códigos documentados	Ping/pong cada 15 s; códigos 1001, 4003, 4008, 4009 y 4029	Cumple	6.5
JWT firmados, contraseñas Argon2id, WebSocket autenticado en el handshake	JWT HS256 de 15 min; Argon2id; ticket de un solo uso de 30 s	Cumple	7.1
Autorización en cada endpoint y mensaje	Participantes, turno y rol administrador	Cumple	7.2
Límites de tasa por usuario y conexión (429 con Retry-After), tamaño y conexiones	Valores de la tabla 7.3	Cumple	7.3
Solo HTTPS y WSS, CORS y orígenes restringidos, cabeceras de seguridad	Ingress sin HTTP, helmet, CORS vacío, Origin ajeno rechazado	Cumple	7.4
Secretos en Key Vault con identidad administrada; ninguno en el repositorio	Referencias a Key Vault; gitleaks en CI	Cumple	7.5
Escaneo de dependencias, código y secretos; imagen no root sin críticas	Dependabot, CodeQL, gitleaks y Trivy; usuario node	Cumple	7.6
Prueba de carga con metas p95 < 200 ms, sin pérdidas ni duplicados, error < 1 %	Bot reproducible; p95 de 15 ms o menos hasta 800 partidas	Cumple	9.1
CPU y memoria por partida y punto de quiebre	0,03 puntos de CPU y 0,02 MB por partida; quiebre ~1 800 jugadas/s	Cumple	9.2
Mensajes compactos	JSON mínimo, 302 bytes de media	Cumple	9.4
Al menos una optimización con números antes y después	Dos, con cifras	Cumple	9.3
Infraestructura como código; ambiente nuevo con una ejecución del pipeline	Bicep; dev y prod creados desde cero por el pipeline	Cumple	10.6

Requisito del enunciado	Cómo se cumple	Estado	Sección
CI/CD con build, linter, pruebas, contratos y escaneo; dev automático, prod manual	GitHub Actions; prod con revisor obligatorio. Dev automático solo si la suscripción lo permite	Cumple parcialmente	10.7 y 11
Publicaciones sin cortar partidas	Revisiones con tráfico al 0 %, drenado y cierre 1001; probado con partidas abiertas	Cumple	11.3
Rollback documentado de menos de 5 minutos	rollback.sh; probado en prod: 22 s	Cumple	11.4
Identidad administrada; sin cadenas de conexión ni claves	Identidad de usuario para ACR, Key Vault y PostgreSQL (Entra)	Cumple	7.5 y 10.5
Observabilidad: OpenTelemetry, logs con id de partida y jugador, métricas propias, sondas, 2 o más alertas	Application Insights; 4 métricas; 2 sondas; 3 alertas	Cumple	12
Costo dentro del crédito, con estimación mensual; dev apagado o a cero	Estimación de ~47 a ~115 USD/mes para prod; dev borrado tras probar	Cumple	14
Repositorio con código, IaC, pipelines, contratos, pruebas, bots y cliente	Todo en el repositorio de GitHub	Cumple	2.2
README: correr en menos de 10 minutos, pruebas y URL pública	README, docker compose up y URL de Azure	Cumple	15
Versión etiquetada v1.0.0 y documentación	Etiqueta v1.0.0 y este documento	Cumple	—

16.2 Rúbrica

Criterio (peso)	Evidencia principal	Estado
Arquitectura y calidad de construcción (20 %)	Dominio puro aislado y verificado por ESLint; capas; TypeScript estricto sin any; decisiones documentadas con diagramas	Cumple
Sesiones y concurrencia (20 %)	Orden por partida, idempotencia, reconexión, abandono, plazos en PostgreSQL, varias réplicas probadas en Azure, publicación con partidas abiertas sin pérdidas	Cumple
Seguridad (20 %)	Autenticación y autorización por partida en HTTP y WebSocket, validación de cada mensaje, límites de tasa, sin filtración al rival, secretos en Key Vault, modelo de amenazas, sin secretos en el repositorio	Cumple
Rendimiento y optimización (15 %)	Prueba de carga reproducible que cumple las metas; punto de quiebre medido; dos optimizaciones con cifras; mediciones en Azure	Cumple
Despliegue en Azure, IaC y CI/CD (10 %)	Bicep, pipeline con OIDC, identidad administrada, prod con aprobación, publicaciones sin cortes, rollback de 22 s, ambiente levantado desde cero	Cumple (dev automático parcial por el límite de la suscripción)
Estándares, contratos y pruebas (10 %)	OpenAPI 3.1 y AsyncAPI 3.0 sincronizados con el código, Problem Details, 294 pruebas en verde en CI	Cumple
Documentación y observabilidad (5 %)	Este documento y docs/; logs y trazas con id de partida y jugador; cuatro métricas propias; sondas; tres alertas	Cumple

17. Limitaciones y trabajo futuro

- **Dev automático:** Azure for Students permite un solo entorno de Container Apps por suscripción, así que dev y prod no pueden coexistir y el despliegue automático a dev quedó apagado. Es el único requisito que cumple parcialmente por una restricción externa.
- **Latencia en Azure:** la carga se hizo desde internet y en escalones pequeños (hasta 8 partidas). La latencia medida en el servidor es baja (media de unos 17 ms), pero el punto de quiebre solo está medido en local y habría que repetirlo dentro de Azure con carga alta.
- **Alertas:** existen y están activas, pero no se ha visto que disparen, y no tienen un grupo de acción por correo (el parámetro alertEmail existe y falta pasarlo en el pipeline).
- **Información oculta:** es opcional en el enunciado; la posición del rival y la trayectoria son públicas por diseño. Sí hay vista por jugador y prueba de no filtración.
- **Polling:** el turno nunca se consulta por polling (todo evento se empuja por WebSocket); el cliente de prueba solo consulta cada cierto tiempo la lista de miembros de una sala privada antes de que empiece la partida, porque el contrato de salas no emite eventos.
- **Navegadores:** el cliente de prueba se probó en Chromium y en emulación móvil; Safari y Firefox no se pudieron probar con navegadores reales, aunque el código se compila para Safari 14 o superior, Firefox 78 y Chrome 87.

- **Mejoras posibles:** medir el punto de quiebre dentro de Azure, revisar las actualizaciones mayores de Dependabot (TypeScript 7, ESLint 10 y Vitest 5) y usar Azure Cache for Redis si el crédito lo permite.

18. Conclusión

El servidor de ASHENFALL cumple lo que pide el proyecto: es autoritativo, seguro y observable; atiende muchas partidas simultáneas e independientes con orden e idempotencia de las jugadas, reconexión y abandono; funciona con varias réplicas coordinadas por PostgreSQL y Redis; y se publica sin cortar partidas, con rollback en segundos. Está desplegado en Azure con infraestructura como código, identidad administrada, secretos en Key Vault y un pipeline con aprobación manual a producción. Las cifras de rendimiento y de resiliencia salen de ejecuciones reales y reproducibles con los bots del repositorio, y donde algo no se pudo comprobar o la suscripción lo impide se dice en este documento.

El servidor está en https://ashenfall-server-prod.icytree-9cfd11e3.canadacentral.azurecontainerapps.io durante el periodo de revisión, y el código, la infraestructura, los pipelines, los contratos, las pruebas, los bots y la documentación están en https://github.com/kevinmirandaa/ashenfall-game, en la versión etiquetada v1.0.0.

Anexo A. El juego que usa el servidor

El enunciado deja el juego a criterio del equipo y solo exige unas restricciones; el juego no tiene nota. Esta sección resume el juego para entender los mensajes del servidor. La historia, las reglas y una guía con capturas están en el README y en docs/guia-de-juego.md.

ASHENFALL es un duelo 1v1 por turnos de proyectiles, de fantasía oscura, que se juega en el navegador. Cada jugador empieza con 100 puntos de vida, el turno dura 30 segundos y la partida tiene como máximo 12 rondas. Se dispara con un gesto de tirachinas (arrastrar hacia atrás para dar potencia y ángulo y soltar). Nadie se mueve por su cuenta: solo se retrocede al recibir un impacto, según el daño. Gana quien deja al rival sin vida; si se acaban las rondas gana quien conserve más vida, y con la misma vida hay empate. Hay siete arenas, y en cada partida el servidor sortea el terreno y los obstáculos. Ganar da 100 Marcas de Ceniza, con las que se compran nueve personajes más en el Arsenal; hay además un modo historia contra diez monstruos que mueve el servidor.

A.1 Restricciones del enunciado

Restricción	Cómo la cumple
Por turnos	El estado guarda el jugador en turno y su plazo; el temporizador de 30 s lo controla el servidor.
Multijugador (mínimo 2 humanos)	Cada partida enfrenta a 2 humanos (cola rápida o sala privada); el modo historia es un extra contra un bot del servidor.
Original	Reglas propias: física de proyectiles determinista, retroceso por impacto, terreno y obstáculos sorteados, personajes comprables.
Reglas verificables	El dominio decide sin ambigüedad si una jugada es válida y cuándo termina la partida; hay pruebas unitarias.
Información oculta (recomendada)	Posición y trayectoria públicas por decisión de diseño; la vista por jugador oculta ángulo, potencia y actionId del rival.



Figura 4. El pueblo de Ashenfall, pantalla principal del cliente de prueba.

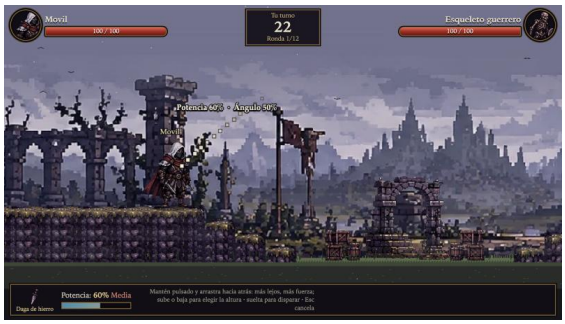


Figura 5. Arena durante un duelo, con la guía de puntería, el turno, la vida y la potencia.